Trabalho 1 – Paralelização do Mandelbrot com OpenMP

Integrante 1 Integrante 2 Integrante 3

Setembro de 2026

Repositório e commit: `[inserir link público e hash]`. O código foi desenvolvido com auxílio do GitHub Copilot para estruturar a versão C, a instrumentação e este rascunho; o grupo revisou e deve conseguir explicar cada decisão.

1 Problema e potencial de paralelização

Calculamos uma imagem do conjunto de Mandelbrot. Para cada pixel (x, y), iteramos $z_{k+1} = z_k^2 + c$ até $|z_k|^2 \geq 4$ ou o limite de 500 iterações. O resultado de um pixel não altera nenhum outro; escolhemos as linhas como unidade de trabalho. A entrada de referência é 1280×720, 500 iterações máximas e [N] quadros, dimensionada para aproximadamente 30–60 s na medição sequencial.

O perfil sequencial com `perf` sobre a mesma região medida indicou que `mandelbrot_iterations`/`render_row` consome [f]% do tempo. Assim, para $P = [\,]$ threads, o limite de Amdahl é $S_{max}(P) = 1/((1-f) + f/P) = [\,]$; o limite assintótico é $1/(1-f) = [\,]$. Partes de janela e CSV ficaram fora tanto do perfil como da medição.

2 Modelagem e diretivas

Em `mandelbrot.c`, `render_image` cria uma região `#pragma omp parallel`; o `#pragma omp for schedule(runtime)` distribui as linhas. Cada thread escreve pixels exclusivos, portanto não há seção crítica. As somas de iterações são locais e unidas por `reduction(+:total_iterations)`. A cláusula `default(none)` obriga declarar explicitamente os dados compartilhados. O caminho `seq` usa os mesmos cálculos sem diretivas e é a referência T_1.

As linhas têm o mesmo número de pixels, mas não necessariamente o mesmo custo: regiões internas ao conjunto atingem o máximo de iterações. Medimos `static`, `dynamic` e/ou `guided` com `-thread-stats`; [justificar a escolha final com os dados].

3 Ambiente e método

Máquina: [modelo, frequência], [núcleos físicos] núcleos físicos e [lógicos] processadores lógicos, L3 de [ ] e [ ] RAM. Sistema: [versão]. Compilador: GCC [versão], flags `-O3 -march=native -fopenmp`. Fixamos as threads com `OMP_NUM_THREADS`. A região medida por `omp_get_wtime()` inclui o cálculo e colorização completos e exclui X11, espera e CSV. Cada configuração foi executada dez vezes; a tabela informa mediana, mínimo e máximo.

Table 1: Escalabilidade forte: entrada fixa [descrever].

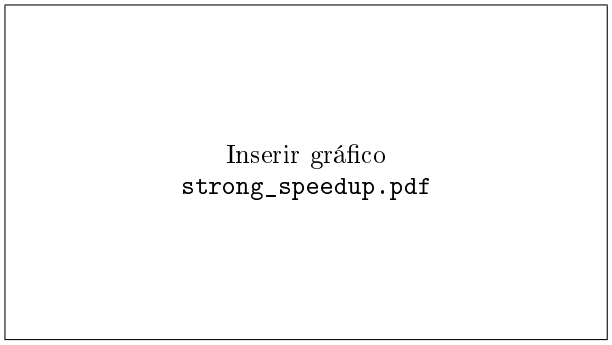
P	mediana (s)	mínimo	máximo	S_P / E_P
1	[ ]	[ ]	[ ]	1.00 / 1.00
2	[ ]	[ ]	[ ]	[ ] / [ ]
4	[ ]	[ ]	[ ]	[ ] / [ ]
[inserir demais linhas]				

Table 2: Escalabilidade fraca. A largura cresce como $1280P$.

P	resolução	mediana (s)	mínimo / máximo	E_{fraca}
1	1280×720	[ ]	[ ] / [ ]	1.00
2	2560×720	[ ]	[ ] / [ ]	[ ]
4	5120×720	[ ]	[ ] / [ ]	[ ]
[inserir demais linhas]				

4 Resultados e análise

Na Figura 1, o speed-up [cresce/satura] a partir de [P] porque [relacionar aos dados: fração sequencial, overhead, desbalanceamento e memória]. A eficiência [ ]. Há [quatro] núcleos físicos e [oito] processadores lógicos; portanto, a



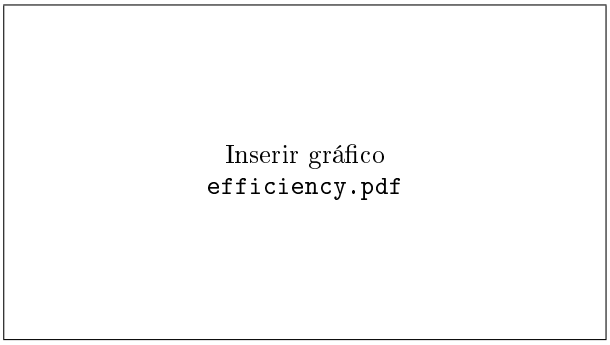


Figure 1: À esquerda, speed-up forte e ideal. À direita, eficiência forte e fraca.

faixa após [4] threads usa SMT. Observamos [efeito mensurado do SMT]. A escalabilidade fraca [se sustenta/não se sustenta] melhor porque [causa baseada na curva]. Os resultados [ficam abaixo/acompanham] o limite de Amdahl por [causa].